

# **TUGAS**

## **VOSA: Sistem E-Voting Pemilihan Kepala Desa Berbasis Kriptografi RSA & SHA256**



**DI SUSUN OLEH:**

**ALIFAH RIZKI SAPUTRI (F1D02310103)**

**ZAHRATU SYITA (F1D02310148)**

**PROGRAM STUDI TEKNIK INFORMATIKA**

**FAKULTAS TEKNIK**

**UNIVERSITAS MATARAM**

**2026**

## **1.1 Latar Belakang**

Pemilihan kepala desa merupakan salah satu wujud demokrasi yang paling dekat dengan kehidupan masyarakat. Secara tradisional, proses ini dilakukan menggunakan kertas suara fisik yang kemudian dihitung secara manual. Cara ini memiliki sejumlah kelemahan: rawan manipulasi pada saat penghitungan, membutuhkan waktu lama, sulit diaudit ulang, dan berpotensi melanggar kerahasiaan apabila pengelolaan kertas suara tidak rapi.

Pemungutan suara secara elektronik (e-voting) menawarkan solusi atas sebagian masalah tersebut, terutama dari sisi kecepatan penghitungan dan kemudahan rekapitulasi atau proses penghitungan suara. Namun, peralihan ke bentuk digital memunculkan risiko baru. Data digital sangat mudah disalin, diubah, dan ditelusuri. Apabila tidak dirancang dengan benar, sebuah sistem e-voting justru dapat membocorkan siapa memilih siapa, atau membuka peluang surat suara dimanipulasi tanpa terdeteksi.

Tantangan inti dari setiap sistem e-voting adalah memenuhi dua kebutuhan yang tampak bertolak belakang secara bersamaan. Di satu sisi, sistem harus dapat memastikan setiap pemilih yang sah hanya memberikan satu suara (akuntabilitas). Di sisi lain, sistem harus menjamin bahwa isi pilihan setiap pemilih tetap rahasia dan tidak dapat dikaitkan kembali dengan identitasnya (kerahasiaan). Menyeimbangkan keduanya tidak dapat dilakukan hanya dengan logika aplikasi biasa diperlukan penerapan teknik kriptografi.

Dari permasalahan itulah dikembangkan VOSA (Voting Suara Aman), sebuah sistem e-voting pemilihan kepala desa yang menempatkan kriptografi sebagai fondasi utamanya. VOSA memanfaatkan enkripsi kunci publik RSA untuk mengamankan isi surat suara, fungsi hash SHA-256 untuk menjaga integritas dan anonimitas, serta perancangan basis data yang secara struktural memisahkan identitas pemilih dari pilihannya.

## **1.2 Tujuan**

Tujuan dari pengembangan sistem VOSA dapat dirinci sebagai berikut:

1. Membangun sistem e-voting pemilihan kepala desa yang menerapkan prinsip kerahasiaan, integritas, dan akuntabilitas suara.
2. Menjamin kerahasiaan pilihan dengan memisahkan identitas pemilih dari isi surat suara secara struktural, sehingga tidak ada pihak yang dapat menelusuri siapa memilih kandidat tertentu.
3. Mengamankan isi surat suara menggunakan enkripsi asimetris RSA sebelum data disimpan ke basis data.
4. Menjaga integritas setiap surat suara menggunakan fungsi hash SHA-256 sehingga perubahan data dapat dideteksi.
5. Mencegah pemungutan suara ganda (double voting) melalui mekanisme penguncian pada tingkat basis data.
6. Memastikan hasil rekapitulasi atau penghitungan suara tidak dapat dibuka sebelum pemilihan resmi ditutup oleh administrator.

### 1.3 Batasan Masalah

Batasan Masalah dari pengembangan sistem VOSA dapat dirinci sebagai berikut:

1. Menggunakan RSA sebagai metode inti, dengan SHA-256 untuk menjaga integritas.
2. Autentikasi pengguna dibuat sederhana (berdasarkan NIK/NIM terdaftar).
3. Belum membahas keamanan level produksi secara penuh (HTTPS, keamanan server, pembagian kunci).
4. Sistem mengelola satu penyelenggaraan pemilihan (satu desa) dengan beberapa kandidat.

### 1.4 Tinjauan Pustaka

#### 1. Kriptografi Kunci Publik dan RSA

Algoritma enkripsi RSA merupakan algoritma asimetris dimana kunci yang digunakan untuk enkripsi disebut kunci publik dan kunci yang digunakan untuk dekripsi disebut kunci privat. Menggunakan algoritma RSA cukup aman karena sulit menghitung bilangan besarsebagai faktor prima. Semakin tinggi angka prima, semakin aman dan baik keamanan data dan informasi. [5]. Algoritma ini banyak diterapkan untuk memberikan layanan kerahasiaan, tanda tangan digital, dan pertukaran kunci [3].

Dalam konteks e-voting, sifat asimetris ini sangat berguna. Sejumlah penelitian menunjukkan bahwa penerapan RSA pada sistem e-voting berhasil mengamankan data suara di dalam basis data sehingga tidak dapat diakses atau diubah oleh pihak yang tidak berwenang [1]. Kunci publik dapat dipakai bebas untuk mengenkripsi setiap suara pada saat pemilih memberikan pilihannya, sementara kunci privat yang menjadi satu-satunya alat untuk membuka suara dapat dijaga ketat dan baru digunakan ketika rekapitulasi dilakukan, sehingga isi surat suara tetap terlindungi selama pemilihan berlangsung [2].

Agar setiap suara yang sama tidak menghasilkan data terenkripsi yang persis sama (yang dapat membuka celah penebakan pola pilihan), proses enkripsi RSA pada VOSA menambahkan unsur acak ke dalam pesan sebelum dienkripsi. Akibatnya, dua suara untuk kandidat yang sama akan tersimpan dalam bentuk yang berbeda di basis data, sehingga kerahasiaan pola pilihan tetap terjaga.

#### 2. Fungsi Hash Kriptografis dan SHA-256

Fungsi hash kriptografis memetakan masukan dengan panjang sembarang menjadi keluaran berukuran tetap (sidik jari digital) yang bersifat satu arah dan praktis tidak dapat dibalik [4]. SHA-256 (Secure Hash Algorithm 256-bit) menghasilkan keluaran sepanjang 256 bit dan banyak digunakan untuk menjamin integritas data, menyimpan informasi sensitif, serta mendukung autentikasi [3]. Salah satu sifat pentingnya adalah avalanche effect, yaitu perubahan kecil pada masukan akan menghasilkan keluaran hash yang sangat berbeda, sehingga setiap modifikasi data mudah terdeteksi [4].

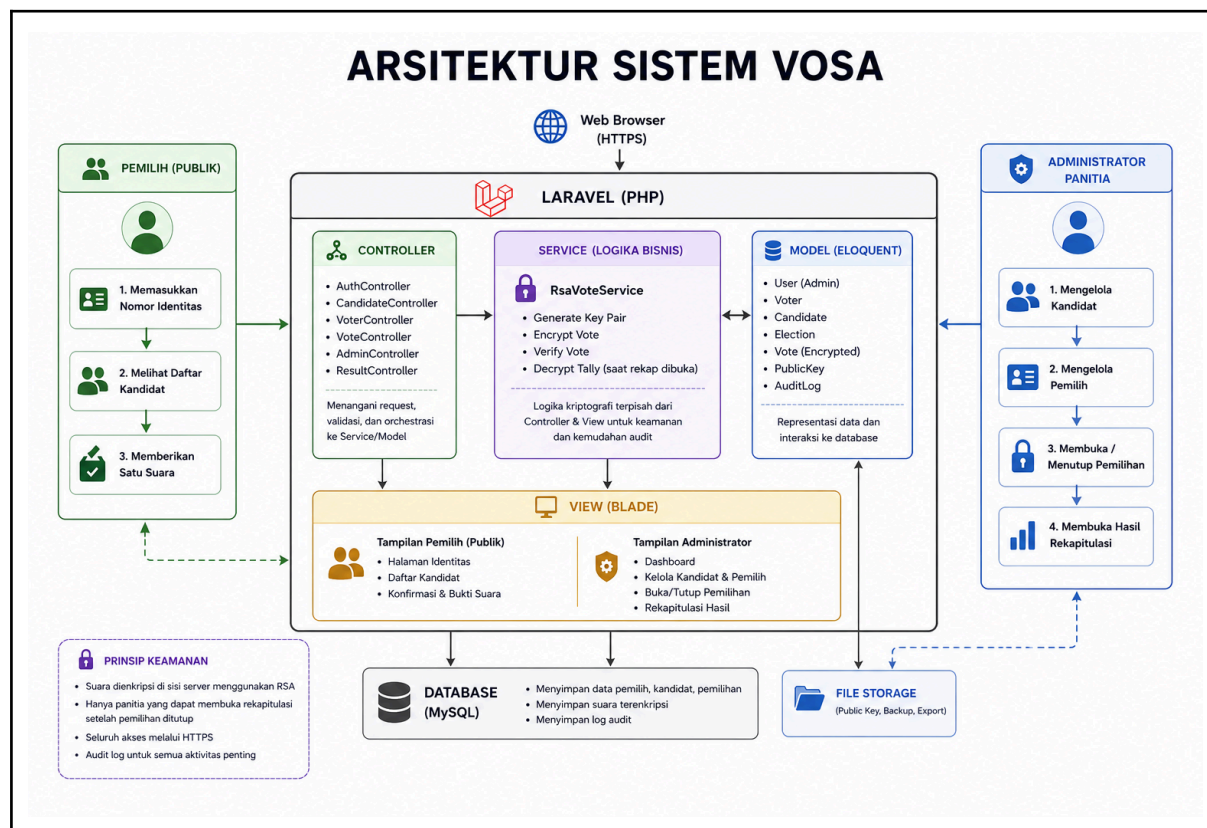
Pada VOSA, SHA-256 dimanfaatkan untuk dua keperluan. Pertama, untuk meng-hash token anonim acak sehingga token aslinya tidak perlu disimpan. Kedua, sebagai dasar

perhitungan nilai integritas pada setiap surat suara, yaitu sebuah nilai yang akan berubah apabila data surat suara dimodifikasi, sehingga setiap perubahan dapat dideteksi.

## 1.5 Perancangan

### 1. Arsitektur Sistem

VOSA dibangun menggunakan kerangka kerja web Laravel (PHP) dengan pola Model-View-Controller (MVC). Logika kriptografi dipusatkan pada sebuah kelas layanan tersendiri (RsaVoteService) agar terpisah dari logika antarmuka dan mudah diaudit. Secara garis besar sistem terbagi atas dua peran pengguna: pemilih (publik) dan administrator panitia.



### 2 Algoritma Kriptografi yang Diterapkan

Tabel berikut merangkum komponen kriptografi yang diterapkan pada sistem beserta perannya:

| Komponen | Peran dalam Sistem                                                                                                                                                             |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RSA      | Mengenkripsi identitas kandidat yang dipilih (isi surat suara). Kunci publik dipakai saat memilih kunci privat hanya dipakai saat rekapitulasi atau proses penghitungan suara. |
| SHA256   | Menghasilkan nilai integritas bagi setiap                                                                                                                                      |

|            |                                                                                                                                                                                                  |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|            | surat suara untuk mendeteksi perubahan data, sekaligus meng-hash token anonim acak agar token asli tidak tersimpan. Serta menyimpan kata sandi administrator dalam bentuk hash, bukan teks asli. |
| Token Acak | Penanda unik bagi setiap surat suara yang dibangkitkan secara acak dan aman, tanpa memuat identitas pemilih.                                                                                     |

### 3. Rancangan Alur Pemungutan Suara

Proses pemungutan satu surat suara dirancang berlangsung dalam satu transaksi basis data agar konsisten dan aman dari kondisi balapan (race condition). Alurnya adalah sebagai berikut:



## 1.6 Source Code

### 1. RsaVoteService.php

#### a. generateKeyPair() /

```
public function generateKeyPair(): array
{
    $opensslConfig = $this->opensslConfig();
    $resource = openssl_pkey_new([
        'private_key_bits' => 2048,
        'private_key_type' => OPENSSL_KEYTYPE_RSA,
```

```

        'config' => $opensslConfig,
    ]);

    if ($resource === false) {
        throw new RuntimeException('Gagal membuat RSA key pair.');
```

```

    }
    if (! openssl_pkey_export($resource, $privateKey, null, ['config'
=> $opensslConfig])) {
        throw new RuntimeException('Gagal mengekspor private key
RSA.');
```

```

    }
    $details = openssl_pkey_get_details($resource);

    if (! is_array($details) || empty($details['key'])) {
        throw new RuntimeException('Gagal membaca public key RSA.');
```

```

    }
    return [
        'public_key' => $details['key'],
        'private_key_encrypted' => Crypt::encryptString($privateKey),
    ];
}

```

Berdasarkan kode di atas, dapat diketahui bahwa method tersebut membuat pasangan kunci RSA yaitu *public key* dan *private key* dengan panjang 2048-bit. Setiap langkah akan di lakukan pengecekan, jika gagal sistem langsung berhenti dan melempar error, jika kunci telah berhasil dibuat, *private key* tidak langsung disimpan begitu saja melainkan di enkripsi dulu menggunakan “Crypt::encryptString()” sebelum masuk ke database agar tidak dapat dibaca langsung sementara *public key* dikembalikan.

b. encryptCandidateId()

```

public function encryptCandidateId(int $candidateId, string
$publicKey): string
{
    $payload = json_encode(['candidate_id' => $candidateId],
JSON_THROW_ON_ERROR);

    if (! openssl_public_encrypt($payload, $encrypted, $publicKey,
OPENSSL_PKCS1_OAEP_PADDING)) {
        throw new RuntimeException('Gagal mengenkripsi suara.');
```

```

    }
    return base64_encode($encrypted);
}

```

Berdasarkan kode di atas, dapat diketahui bahwa method tersebut mengenkripsi pilihan suara pemilih, ID calon atau kandidat dikemas dulu ke format JSON lalu di enkripsi menggunakan *public key* RSA dengan metode OAEP yang merupakan metode enkripsi asimetris yang aman dan tahan serangan. Hasil enkripsinya berupa data biner, lalu dikonversi ke Base64 agar bisa disimpan sebagai teks di database.

c. decryptCandidateId()

```

public function decryptCandidateId(Ballot $ballot, RsaKey $rsaKey):
int
{
    $privateKey =

```

```

Crypt::decryptString($rsaKey->private_key_encrypted);
    $privateKeyResource = openssl_pkey_get_private($privateKey);
    if ($privateKeyResource === false) {
        throw new RuntimeException('Private key RSA tidak valid untuk
dekripsi.');
```

```

    }
    if (!
openssl_private_decrypt(base64_decode($ballot->encrypted_vote),
$decrypted, $privateKeyResource, OPENSSL_PKCS1_OAEP_PADDING)) {
        throw new RuntimeException('Gagal mendekripsi ballot.');
```

```

    }
    $payload = json_decode($decrypted, true, flags:
JSON_THROW_ON_ERROR);

    return (int) $payload['candidate_id'];
}

```

Berdasarkan kode di atas, dapat diketahui bahwa method tersebut kebalikan dari enkripsi, digunakan saat admin membuka rekap suara setelah pemilihan ditutup. Pertama *private key* didekripsi dulu dari database, kemudian digunakan untuk membuka suara terenkripsi, hanya *private key* yang pasangannya cocok dengan *public key* yang bisa membuka enkripsi tersebut. Setelah berhasil didekripsi, hasilnya di-parse dari JSON dan dikembalikan sebagai ID calon yang dipilih.

#### d. integrityHash()

```

public function integrityHash(string $anonymousTokenHash, string
$encryptedVote): string
{
    return hash_hmac('sha256',
$anonymousTokenHash.'|'.$encryptedVote, config('app.key'));
}

```

Berdasarkan kode di atas, dapat diketahui bahwa method tersebut membuat sidik jari unik untuk setiap surat suara menggunakan algoritma HMAC-SHA256 dengan menggabungkan token anonim dan suara terenkripsi, lalu di-hash menggunakan APP\_KEY sebagai kunci rahasia. Tujuannya untuk mendeteksi apabila ada data surat suara yang diubah di database.

## 2. PublicVotingController.php

#### a. lookup()

```

public function lookup(Request $request): RedirectResponse
{
    $data = $request->validate([
        'identity_number' => ['required', 'string', 'max:40'],
    ]);

    $voter = Voter::where('identity_number',
strtoupper(trim($data['identity_number']))->first();
    if (!$voter) {
        return back()->withErrors(['identity_number' => 'NIK/NIM
tidak terdaftar sebagai pemilih.'])->withInput();
    }
    if ($voter->has_voted) {
        return back()->withErrors(['identity_number' => 'Anda sudah

```

```

menggunakan hak pilih.'])->withInput();
    }
    session(['voter_id' => $voter->id]);

    return redirect()->route('vote.show');
}

```

Berdasarkan kode di atas, dapat diketahui bahwa method tersebut memproses *input* NIK dari pemilih. Pertama input divalidasi, lalu NIK dikonversi ke huruf kapital menggunakan “`strtoupper()`” dan dihapus spasinya menggunakan “`trim()`” sebelum dicari di database. Terdapat dua pengecekan keamanan yaitu apakah NIK/NIM terdaftar dan apakah pemilih sudah pernah memilih. Jika lolos keduanya, ID pemilih disimpan ke session sebagai penanda sesi voting aktif, lalu diarahkan ke halaman pilih calon.

#### b. submit()

```

public function submit(Request $request, RsaVoteService $rsa):
RedirectResponse
{
    $voter = $this->currentVoter();

    if (! $voter) {
        return redirect()->route('home');
    }

    $data = $request->validate([
        'candidate_id' => ['required', 'integer',
'exists:candidates,id'],
    ]);

    $selection = Election::with('rsaKey')->firstOrFail();
    $candidate = Candidate::where('election_id',
$selection->id)->findOrFail($data['candidate_id']);

    if (! $selection->isOpen()) {
        return back()->withErrors(['candidate_id' => 'Pemilihan sudah
ditutup.']);
    }

    DB::transaction(function () use ($voter, $candidate, $selection,
$rs): void {
        $lockedVoter =
Voter::whereKey($voter->id)->lockForUpdate()->firstOrFail();

        if ($lockedVoter->has_voted) {
            abort(409, 'Hak pilih sudah digunakan.');
```



```

        'encrypted_vote' => $encryptedVote,
        'integrity_hash' =>
$rsa->integrityHash($anonymousTokenHash, $encryptedVote),
        'cast_at' => now(),
    ]);

    $lockedVoter->update([
        'has_voted' => true,
        'voted_at' => now(),
    ]);
});

session()->forget('voter_id');

return redirect()->route('vote.success');
}

```

Berdasarkan kode di atas, terdapat beberapa tahap yang dilakukan. Pertama sistem memvalidasi bahwa kandidat yang dipilih benar-benar ada dan pemilihan masih dalam status terbuka. Selanjutnya seluruh proses penyimpanan suara dibungkus dalam DB::transaction() untuk menjamin bahwa semua operasi database berhasil atau tidak. Data pemilih dikunci menggunakan lockForUpdate() untuk mencegah double voting yang mungkin terjadi akibat pengiriman request secara bersamaan. Kemudian dibuat token anonim acak sepanjang 32 byte menggunakan random\_bytes(), lalu di-hash menggunakan SHA-256 sehingga token aslinya tidak pernah tersimpan di database. Pilihan suara dienkripsi menggunakan encryptCandidateId() dari RsaVoteService, lalu disimpan ke tabel ballots bersama hash integritas. Setelah surat suara tersimpan, status pemilih diperbarui menjadi has\_voted = true. Terakhir, sesi voting dihapus dan pemilih diarahkan ke halaman sukses.

### 3. AdminController.php

#### a. tally()

```

public function tally(RsaVoteService $rsa): View
{
    $selection = Election::with(['candidates',
'rsaKey'])->withCount('ballots')->firstOrFail();
    $results = $selection->candidates->mapWithKeys(fn (Candidate
$candidate) => [$candidate->id => 0])->all();

    if ($selection->status === 'closed') {
        foreach ($selection->ballots as $ballot) {
            $candidateId = $rsa->decryptCandidateId($ballot,
$selection->rsaKey);
            $results[$candidateId] = ($results[$candidateId] ?? 0) +
1;
        }
    }

    $totalVoters = Voter::count();

    return view('admin.tally', compact('election', 'results',
'totalVoters'));
}

```

Berdasarkan kode di atas, method tally() bertugas menghitung hasil akhir suara. Pertama semua data pemilihan beserta kandidat dan kunci RSA diambil dari database. Sistem kemudian mengecek apakah status pemilihan sudah closed, karena hasil suara hanya boleh dibuka setelah pemilihan ditutup. Jika sudah ditutup, setiap surat suara yang tersimpan di database didekripsi satu per satu menggunakan decryptCandidateId() untuk mendapatkan ID kandidat yang dipilih, lalu jumlah suara tiap kandidat dihitung dan dikirim ke halaman rekap.

b. updateElectionStatus()

```
public function updateElectionStatus(Request $request):
RedirectResponse
{
    $data = $request->validate([
        'status' => ['required', 'in:open,closed'],
    ]);

    $selection = Election::firstOrFail();
    $selection->update([
        'status' => $data['status'],
        'started_at' => $data['status'] === 'open' ?
($selection->started_at ?? now()) : $selection->started_at,
        'ended_at' => $data['status'] === 'closed' ? now() : null,
    ]);

    $this->log($data['status'] === 'open' ? 'Buka pemilihan' : 'Tutup
pemilihan', 'Status pemilihan diperbarui.');
```

```
        return back()->with('status', 'Status pemilihan berhasil
diperbarui.');
```

```
    }
```

Berdasarkan kode di atas, method ini mengatur status pemilihan menjadi open atau closed. Input status divalidasi terlebih dahulu agar hanya dua nilai tersebut yang diterima. Ketika pemilihan dibuka, started\_at diisi dengan waktu saat ini jika belum pernah diisi sebelumnya. Ketika pemilihan ditutup, ended\_at diisi dengan waktu penutupan dan secara otomatis mengunci seluruh proses voting dari sisi pemilih. Setiap perubahan status juga dicatat ke audit log melalui method log().

c. live()

```
public function live(Request $request, RsaVoteService $rsa): View
{
    $selection = Election::with(['candidates', 'rsaKey',
'ballots'])->withCount('ballots')->firstOrFail();
    $voterQuery = Voter::query();
    if ($request->filled('region')) {
        $region = $request->string('region');
        $voterQuery->where('region', $region);}

    $totalVoters = (clone $voterQuery)->count();
    $votedCount = (clone $voterQuery)->where('has_voted',
true)->count();
    $notVotedCount = max($totalVoters - $votedCount, 0);
    $turnout = $totalVoters > 0 ? round(($votedCount / $totalVoters)
```

```

* 100, 1) : 0;
$results = collect();

if ($selection->status === 'closed') {
    $counts = $selection->candidates->mapWithKeys(fn (Candidate
$candidate) => [$candidate->id => 0])->all();

    foreach ($selection->ballots as $ballot) {
        $candidateId = $rsa->decryptCandidateId($ballot,
$selection->rsaKey);
        $counts[$candidateId] = ($counts[$candidateId] ?? 0) + 1;
    }

    $results = $selection->candidates
    ->map(fn (Candidate $candidate) => [
        'candidate' => $candidate,
        'votes' => $counts[$candidate->id] ?? 0,
    ])
    ->sortByDesc('votes')
    ->values(); }

$regions =
Voter::select('region')->distinct()->orderBy('region')->pluck('region
');

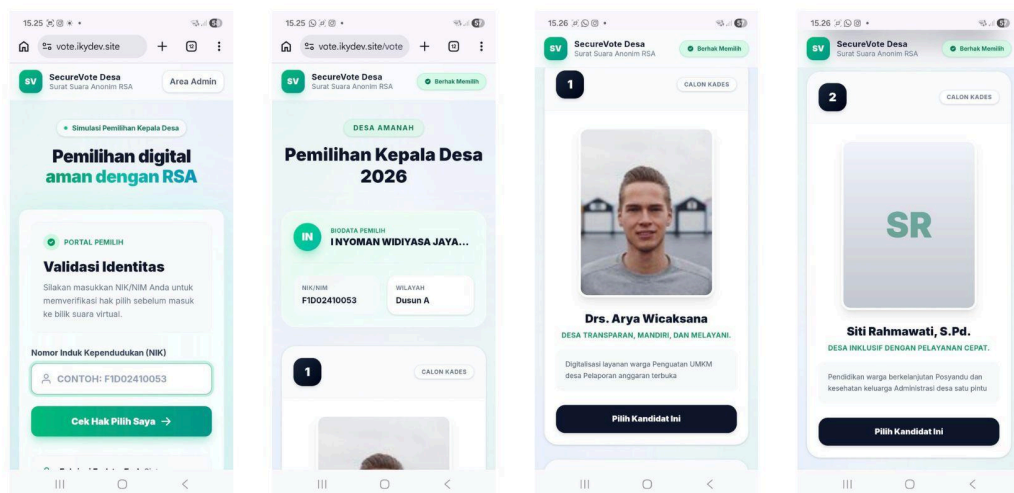
return view('admin.live', compact('election', 'totalVoters',
'votedCount', 'notVotedCount', 'turnout', 'results', 'regions'));}

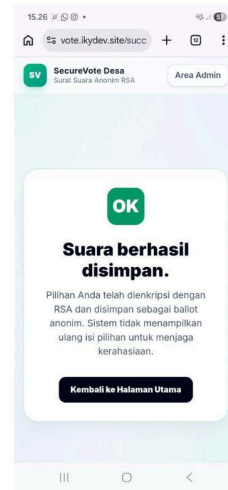
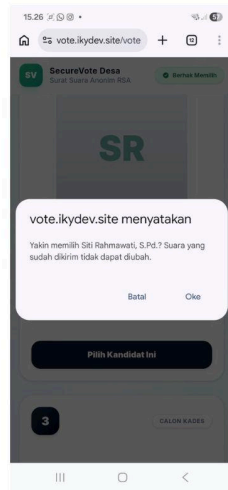
```

Berdasarkan kode di atas, method live() menampilkan pantauan partisipasi pemilih secara langsung. Admin dapat memfilter data berdasarkan wilayah menggunakan parameter region dari request. Sistem menghitung total pemilih, jumlah yang sudah memilih, yang belum memilih, serta persentase partisipasi. Selama pemilihan masih berlangsung, hasil suara per kandidat tidak ditampilkan. Hasil baru akan muncul setelah status pemilihan berubah menjadi closed, dimana setiap surat suara didekripsi dan diurutkan dari kandidat dengan suara terbanyak.

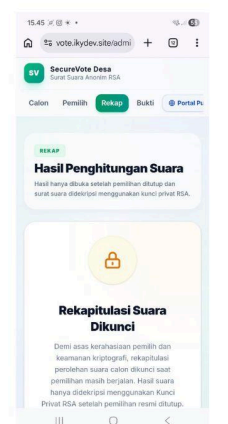
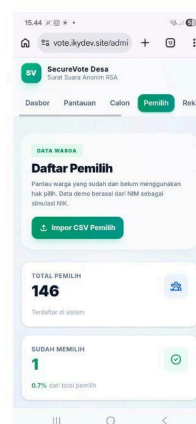
## 1.7 Tampilan dan Demo sistem

### a. Sisi Client





## b. sisi admin





c. akses web (<https://vote.ikydev.site>)

## DAFTAR PUSTAKA

- [1] Jurnal Informatika dan Teknik Elektro Terapan (JITET), "Implementasi Algoritma RSA pada Aplikasi E-Voting (Studi Kasus: Organisasi Mahasiswa UMKT)," Univ. Lampung, 2024.
- [2] M. Ridwan, dkk., "Rancang Bangun E-Voting dengan Menggunakan Keamanan Algoritma Rivest Shamir Adleman (RSA) Berbasis Web (Studi Kasus: Pemilihan Ketua BEM FMIPA)," Informatika Mulawarman: Jurnal Ilmiah Ilmu Komputer, vol. 11, no. 2, 2016.
- [3] Jurnal Teknologi Informasi dan Ilmu Komputer (JTIK), "Keamanan Data Menggunakan Secure Hashing Algorithm (SHA)-256 dan Rivest Shamir Adleman (RSA) pada Digital Signature," vol. 10, no. 6, hlm. 1213–1222, 2023.
- [4] S. N. Neyman, M. F. Isnaini, dan S. Nurdianti, "Penerapan Sistem E-Voting pada Pemilihan," Jurnal Sains Terapan, edisi III, vol. 3, no. 1, hlm. 35–49, 2013.
- [5] M. S. Dairi dan sitorus, "Rancang Bangun Website Pengaman Database E-Voting dengan Menerapkan Algoritma Rivest Shamir Adleman (RSA)" Jurnal Teknolgi Informatika dan Komputer (JTIK), 2023..